Software Requirements Specification

Site Doctor

An Agentic Website Auditing and Remediation System

Version 1.0

July 23, 2026

Document Status	Draft
Version	1.0
Prepared For	Site Doctor Project (Portfolio / Academic Project)
Standard Followed	IEEE 830-1998 SRS Structure (adapted)

Contents

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of **Site Doctor**, an agentic system that crawls a given website, evaluates it across Search Engine Optimization (SEO), usability/conversion design, and (optionally, with explicit authorization) passive security posture, and produces a ranked, human-readable report. For a defined subset of mechanically verifiable issues, the system additionally proposes concrete fixes, applies them only after explicit human approval, and re-verifies that each applied fix actually resolved the underlying issue before reporting it as resolved.

This document is intended for the developer(s) implementing the system, and any reviewer or instructor evaluating the project's scope and design rationale. It is the baseline against which the Software Design Document (SDD), Use Case Specification, UML diagrams, and Database Design Document are produced.

1.2 Scope

Site Doctor operates as a multi-agent pipeline orchestrated with LangGraph. Given a target URL and a human-selected set of checks (SEO, UX, and/or Security), the system:

(a) Crawls and renders the target page using a headless browser (Playwright), producing a local HTML copy and, where UX review is selected, a set of scroll-position screenshots.

(b) Runs a rule-based, mechanically verifiable audit (Google Lighthouse) covering SEO, accessibility, and performance, only when SEO is selected.

(c) Runs a vision-model-based usability and conversion review over the captured screenshots, only when UX is selected.

(d) Runs a passive, non-intrusive security posture check (HTTP security headers, TLS/certificate configuration, exposed software-version/CVE lookups) only when the user explicitly opts in, and only for a site the user has confirmed they are authorized to test. Active scanning, exploitation, or load/stress testing is explicitly **out of scope** for this system.

(e) Aggregates and ranks findings from whichever checks were run into a single report, separating mechanically verifiable *Issues* from judgment-based *UX Suggestions*.

(f) For a defined subset of high-confidence, mechanically verifiable issues, generates a proposed fix, presents a before/after diff to the user, and applies it only on explicit approval.

(g) Re-runs the relevant audit against the patched copy to verify the fix actually cleared, looping back to fix generation on failure up to a configured retry limit.

The system does *not*, in this version, perform live write-back to a user's production site, perform authenticated crawling, crawl beyond a single page, or perform any form of active security testing.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification

Term	Definition
SEO	Search Engine Optimization
UX	User Experience
Issue	A mechanically verifiable finding produced by a rule-based audit tool (e.g. Lighthouse), possessing a deterministic pass/fail check.
UX Suggestion	A judgment-based finding produced by the vision-review agent. Has no deterministic re-check and is presented to the human as a recommendation, not auto-applied.
Fix	A concrete, proposed code-level change (e.g. new meta description, alt text) generated for a specific Issue.
Human-in-the-loop (HITL)	A workflow checkpoint at which the system pauses execution and requires explicit human input or approval before proceeding.
LangGraph	The graph-based agent orchestration framework used to implement the system's multi-agent pipeline, including parallel (fan-out/fan-in) and conditional execution.
Lighthouse	Google's open-source, automated tool for auditing web page quality across performance, accessibility, SEO, and best-practice categories.
Playwright	A browser automation framework used for headless page rendering and screenshot capture.
LLM	Large Language Model.
CVE	Common Vulnerabilities and Exposures — a public identifier for a known software security vulnerability.

1.4 References

]

[1 IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications.*

[2 Google Lighthouse Documentation – https://developer.chrome.com/docs/lighthouse

[3 Playwright Documentation – https://playwright.dev

[4 LangGraph Documentation – https://langchain-ai.github.io/langgraph/

[5 OpenAI API Documentation – https://platform.openai.com/docs

1.5 Document Overview

Section 2 describes the product context, functions, users, and general constraints at a high level. Section 3 enumerates specific functional requirements. Section 4 enumerates non-functional requirements. Section 5 describes external interfaces. Section 6 lists design and implementation constraints. Section 7 lists assumptions and dependencies. Section 8 contains supporting appendices.

2 Overall Description

2.1 Product Perspective

Site Doctor is a new, self-contained system; it is not a component of, or replacement for, any existing product. It composes several independent third-party tools (Playwright, Lighthouse, an OpenAI-compatible LLM API) behind a single orchestrated pipeline, exposing a report to the end user. It is designed initially as a locally-run command-line/script-driven tool, with a lightweight (e.g. Streamlit) interface as a near-term extension, and is architected so that individual audit capabilities (SEO, UX, Security) can be toggled independently per run.

2.2 Product Functions

At a high level, the system provides the following functions:

- Accept a target URL and a human-selected set of checks to run.
- Render the target page and capture the artifacts each selected check requires (HTML copy; scroll-position screenshots).
- Execute the selected audits, independently and in parallel where more than one is selected.
- Aggregate findings into a single ranked report, with mechanically verifiable Issues and judgment-based UX Suggestions kept distinct.
- Propose fixes for a defined subset of Issues.
- Require explicit human approval before applying any fix.
- Re-verify applied fixes against the same audit tool that originally flagged them, retrying up to a configured limit.
- Present a final report summarizing what was found, what was fixed and verified, and what remains outstanding.

2.3 User Characteristics

The primary user is assumed to be a website owner, freelancer, or developer with basic technical literacy (able to read a plain-language issue description and approve/reject a proposed change) but not necessarily deep expertise in SEO, accessibility standards, or web security. The system is designed to explain findings in plain language rather than raw technical audit output.

2.4 General Constraints

- The system depends on three external services/tools it does not control: the Lighthouse CLI (Node.js), Playwright/Chromium, and an LLM API (OpenAI). Availability, rate limits, and pricing changes to any of these are outside this system's control.
- The Security check is opt-in, defaulted to *off*, and limited to passive, non-intrusive checks. The system is not authorized, and is explicitly designed not to attempt, active vulnerability exploitation or load/stress testing against any target.

- Fix application operates only against a local, offline copy of the crawled page. The system does not perform live write-back to a production site or content management system in this version.

2.5 Assumptions and Dependencies

See Section 7.

3 Functional Requirements

Each requirement is assigned a unique identifier, a priority (**M** = Mandatory, **S** = Should-have, **C** = Could-have), and is independently verifiable.

3.1 URL Input and Validation

ID	Requirement	Pri.
FR-01	The system shall accept a website URL as input from the user.	M
FR-02	The system shall validate that the input is a well-formed, reachable HTTP/HTTPS URL before proceeding, and shall return a clear error message if validation fails.	M
FR-03	The system shall present the user with a selectable set of checks to run (SEO, UX, Security) before crawling begins.	M
FR-04	The Security check option shall be unselected (off) by default and shall require explicit, affirmative user selection to run.	M

3.2 Crawling and Artifact Capture

ID	Requirement	Pri.
FR-05	The system shall render the target page using a headless browser and save a local, self-contained HTML copy, regardless of which checks are selected.	M
FR-06	The system shall capture a sequence of scroll-position screenshots of the rendered page *only if* the UX check is selected.	M
FR-07	The screenshot sequence shall include an above-the-fold capture (no scroll) and up to a configurable maximum number of additional captures at successive scroll positions.	S

3.3 SEO / Rule-Based Audit

ID	Requirement	Pri.
FR-08	The system shall execute a Lighthouse audit against the target URL when the SEO check is selected, covering SEO, Accessibility, and Performance categories.	M
FR-09	The system shall parse the Lighthouse report and extract only the category scores and the specific failing audit items relevant to a predefined, trackable set of checks, rather than retaining the full raw report.	M
FR-10	Each extracted finding shall be represented as a structured Issue object with a stable identifier, category, title, description, and (once triaged) a severity and plain-language summary.	M

3.4 UX / Vision-Based Review

ID	Requirement	Pri.
FR-11	The system shall submit the captured screenshot sequence to a vision-capable LLM in a single request when the UX check is selected, so that findings can account for scroll continuity across the page.	M
FR-12	The system shall instruct the model that screenshots represent the same page in scroll order, to avoid duplicate or contradictory findings across images.	M
FR-13	Each finding shall be represented as a structured UX Suggestion object with an identifier, category, severity, observation, and recommendation, and shall be kept in a data structure distinct from mechanically verifiable Issues.	M

3.5 Security / Passive Posture Check

ID	Requirement	Pri.
FR-14	The system shall only execute the Security check when explicitly selected by the user for the current run.	M
FR-15	The Security check shall be limited to passive, non-intrusive checks: presence/absence of standard HTTP security headers, TLS/certificate configuration, and lookup of publicly known CVEs for any software versions exposed by the target.	M
FR-16	The system shall not perform active vulnerability exploitation, authentication bypass attempts, or load/stress testing against any target under any configuration.	M

3.6 Triage, Ranking, and Reporting

ID	Requirement	Pri.
FR-17	The system shall wait for all selected audit branches to complete before proceeding to triage (i.e. triage shall not run on a partial result set).	M
FR-18	The system shall rank mechanically verifiable Issues by severity and produce a plain-language summary for each, suitable for a non-technical reader.	M
FR-19	The system shall present Issues and UX Suggestions as clearly separated sections in the final report, and shall not merge them into a single undifferentiated list.	M
FR-20	The system shall allow the user to view or download the generated report.	S

3.7 Fix Generation, Approval, and Verification

ID	Requirement	Pri.
FR-21	The system shall generate a proposed Fix only for Issues that are mechanically verifiable; UX Suggestions shall never be auto-applied.	M
FR-22	Each proposed Fix shall include an explicit before/after representation of the change.	M
FR-23	The system shall present each proposed Fix to the user and shall not apply any Fix without explicit, per-fix (or explicit batch) approval.	M
FR-24	Approved Fixes shall be applied only to the local copy of the crawled page, not to any live/production system.	M
FR-25	After applying approved Fixes, the system shall re-run the originating audit against the patched local copy and mark each Fix as verified-cleared or not.	M
FR-26	If a Fix does not verify as cleared, the system shall route back to fix generation for that Issue, up to a configurable maximum number of retries, after which it shall report the Issue as unresolved with the attempted history.	S

4 Non-Functional Requirements

4.1 Performance

ID	Requirement	Pri.
NFR-01	An SEO-only audit run shall complete within 2 minutes under normal network conditions for a typical single-page site.	S

ID	Requirement	Pri.
NFR-02	A combined SEO + UX audit run shall complete within a target window appropriate to the added vision-model latency; this target shall be measured and documented empirically once the pipeline is complete, rather than fixed a priori, since it depends on external LLM API latency outside this system's control.	C
NFR-03	The system shall avoid sending full, unfiltered raw tool output (e.g. complete Lighthouse JSON reports) to any LLM; only extracted, structured, relevant fields shall be included in LLM prompts, to control both latency and cost.	M

4.2 Scalability

ID	Requirement	Pri.
NFR-04	The initial version targets single-user, single-run local execution. Multi-concurrent-user support (e.g. 100 concurrent users) is identified as a future goal contingent on migrating to a hosted, queue-backed deployment, and is out of scope for the version described by this SRS.	C

4.3 Reliability and Availability

ID	Requirement	Pri.
NFR-05	Uptime targets (e.g. 99%) apply only once the system is deployed as a hosted service; they are not applicable to the local-execution version described by this SRS and shall be re-specified at that stage.	C
NFR-06	The system shall fail gracefully and report a clear error when an external dependency (Lighthouse CLI, Playwright/Chromium, or the LLM API) is unavailable, rather than crashing without explanation.	M

4.4 Security

ID	Requirement	Pri.
NFR-07	All API communication with the LLM provider shall occur over HTTPS.	M
NFR-08	API keys and credentials shall be supplied via environment variables and shall not be hard-coded or committed to source control.	M

ID	Requirement	Pri.
NFR-09	The Security check module itself shall be restricted to passive analysis only, as specified in FR-15/FR-16, to avoid the system becoming a source of unauthorized scanning activity.	M

4.5 Usability

ID	Requirement	Pri.
NFR-10	All Issue and UX Suggestion descriptions surfaced to the user shall be written in plain, non-technical language.	M
NFR-11	The interface (command-line or UI) shall clearly distinguish mechanically-verified findings from judgment-based suggestions at all times, per FR-19.	M
NFR-12	Once a UI is implemented, it shall be responsive across common desktop viewport sizes; mobile responsiveness is a Could-have for the initial version.	C

4.6 Maintainability

ID	Requirement	Pri.
NFR-13	All cross-node data shall be defined via explicit, typed schema objects (e.g. Pydantic models) rather than untyped dictionaries, to keep the multi-agent graph's state debuggable.	M
NFR-14	Each audit capability (SEO, UX, Security) shall be implemented as an independently testable module invocable in isolation from the full graph, to support incremental development and debugging.	M

5 External Interface Requirements

5.1 User Interfaces

The initial version exposes a command-line interface accepting a URL and a check-selection prompt. A subsequent lightweight web UI (e.g. Streamlit) is planned to present the same inputs via selectable checkboxes and to render the final report, including before/after diffs for proposed fixes and an approve/reject control per fix.

5.2 Hardware Interfaces

None beyond a standard workstation capable of running a headless Chromium instance (via Playwright) and a Node.js runtime (for the Lighthouse CLI).

5.3 Software Interfaces

Interface	Description
Playwright / Chromium	Used for headless page rendering, HTML capture, and scroll-position screenshot capture.
Lighthouse CLI (Node.js)	Invoked as a subprocess; JSON report is parsed and filtered before any further processing.
OpenAI-compatible LLM API	Used for: (a) triage/plain-language summarization of Issues, (b) vision-based UX review over screenshots, and (c) fix generation for approved Issue types. Communicated over HTTPS.
CVE / vulnerability lookup source	Used by the passive Security check to cross-reference exposed software versions against publicly known vulnerabilities.

5.4 Communication Interfaces

All external API communication (LLM provider, CVE lookup source) occurs over HTTPS. No inbound network interface is exposed by the system itself in its initial local-execution form.

6 Constraints

- **Legal/Ethical Constraint:** The Security check shall only be run against targets the user has confirmed they are authorized to test, shall default to off, and shall never perform active exploitation or load/stress testing, per FR-14 through FR-16. This constraint is non-negotiable and takes precedence over any feature request that would violate it.
- **Single-page scope:** Crawling in this version is limited to a single target page; multi-page/full-site crawling is out of scope.
- **No live write-back:** Fix application operates only on a local copy; direct modification of a user's live production site or CMS is out of scope for this version.
- **Third-party dependency risk:** Lighthouse, Playwright, and the LLM API are external dependencies whose behavior, pricing, or availability the system does not control.
- **Cost constraint:** LLM calls shall operate on filtered/extracted data (per NFR-03), and the number of screenshots submitted per UX review is capped, to keep per-run cost bounded and predictable.

7 Assumptions and Dependencies

- It is assumed the target URL is publicly reachable without authentication.
- It is assumed the user has, or will obtain, valid API credentials for the LLM provider used.
- It is assumed Node.js and the Lighthouse CLI (or npx-based fallback) are installed and reachable on the executing machine's PATH.

- It is assumed that, for any run with the Security check enabled, the user has independently confirmed they hold authorization to test the target site; the system does not and cannot verify this authorization itself, and relies on the user's explicit opt-in as the control point.
- It is assumed that Lighthouse's headless Chromium execution mode is compatible with the host operating system; known incompatibilities (e.g. legacy headless mode issues on some Windows configurations) are mitigated by using the `--headless=new` flag.

8 Appendix

8.1 Appendix A: High-Level Pipeline Overview

```
check_selection (human input: which checks to run)
        |
       crawl (HTML always; screenshots only if UX selected)
        |
  conditional fan-out based on selection
        |
   +----+----+---------------+
   v         v               v
seo_audit  ux_review   security_audit   (only selected branches run)
   |         |               |
   +----+----+---------------+
        |
      triage (ranks Issues; UX Suggestions pass through)
        |
       fix (proposed changes for verifiable Issues only)
        |
     approve (human-in-the-loop)
        |
      apply (writes to LOCAL copy only)
        |
     reaudit (re-verify against same tool that found the issue)
        |
   verified? --no--> back to fix (up to max retries)
        |
       yes
        |
     report
```

8.2 Appendix B: Requirement Priority Key

- **M — Mandatory:** Required for the system to be considered functionally complete for this version.
- **S — Should-have:** Important but not blocking for an initial working version.
- **C — Could-have:** Desirable future extension, explicitly out of scope for the current version's completion criteria.

8.3 Appendix C: Revision History

Version	Date	Description
1.0	July 23, 2026	Initial SRS draft covering SEO, UX, and opt-in passive Security auditing pipeline with human-in-the-loop fix approval.